



Diseño General del Sistema – POWERPLAY discomóvil

⚙ Estado	Completado
👤 Propietario	Ⓜ Raúl LeciñenaⓂ María Paradela🇪🇸 Oscar Cerlanga
☰ Categoría	
☰ Equipo	

1. Visión general del diseño

POWERPLAY discomóvil se diseña como una aplicación web de gestión de reservas para eventos, basada en una arquitectura cliente-servidor y apoyada en una base de datos relacional. El sistema permite gestionar usuarios, reservas, recursos (equipos y artistas), presupuestos, facturas, pagos, eventos públicos y formularios de contacto, garantizando la trazabilidad completa desde la creación de una reserva hasta el registro de los pagos asociados.

El diseño del sistema se apoya en un modelo de datos centralizado, integra la pasarela de pagos Stripe para el cobro online, el motor de automatizaciones N8N para el envío de correos electrónicos, Supabase Storage para la gestión de imágenes y logotipos, y la API de Google Maps para la asistencia en la introducción de direcciones y la visualización de ubicaciones.

2. Arquitectura general del sistema

El sistema se compone de los siguientes bloques principales:

- **Cliente web (Front-end):** interfaz desarrollada en React + Vite, accesible desde navegador. Gestiona la interacción del usuario, la presentación de datos, los pagos con Stripe Elements y la geolocalización con Google Maps Places API.
- **Servidor de aplicación (Back-end):** aplicación Node.js + Express que implementa la lógica de negocio, gestiona las operaciones sobre todas las

entidades, valida roles de acceso, calcula disponibilidad de recursos por fechas y orquesta las llamadas a servicios externos.

- **Base de datos relacional:** PostgreSQL desplegado en Supabase. Almacena la información estructurada del sistema con RLS habilitado en todas las tablas.
 - **Stripe:** pasarela de pagos para el cobro online mediante tarjeta. El backend crea PaymentIntents y procesa los resultados mediante webhook firmado con verificación de firma y comprobación de idempotencia.
 - **N8N:** motor de automatizaciones alojado en Hostinger. Ejecuta 8 flujos de trabajo activados por webhooks POST del backend para el envío de correos HTML a clientes y administradores en cada evento clave del sistema.
 - **Supabase Storage:** almacenamiento de imágenes de equipos, DJs y logotipo de empresa. Las subidas se gestionan desde el backend con la `SERVICE_ROLE_KEY` para evitar restricciones de RLS.
 - **Google Maps API:** Places API para autocompletado de direcciones en formularios y visualización de ubicaciones en la página de eventos.
-

3. Diseño basado en el modelo de datos

El diseño funcional del sistema gira en torno a las siguientes entidades:

- **Usuario:** clientes del sistema con rol (usuario/admin) y dirección completa almacenada.
- **Reserva:** solicitud de servicio para una fecha y ubicación. Almacena los datos del cliente de forma directa (desnormalización intencional).
- **Equipo y DJ:** recursos asociables a una reserva. Equipo incorpora `stock_total` y `categoria`; DJ incorpora `tipo` (dj, orquesta, grupo).
- **Presupuesto:** propuesta económica con estado restringido (pendiente → aceptado_cliente → aceptado / rechazado) y fecha límite de validez.
- **Detalle_Presupuesto:** líneas de concepto, cantidad y precio unitario del presupuesto.
- **Factura:** documento legal generado automáticamente al confirmar el presupuesto. Numeración correlativa `FAC-YYYY-XXXX`.

- **Pago:** registro de pagos parciales o totales. Admite Stripe, efectivo y transferencia.
 - **Contacto:** formulario público con tipo restringido, estado de respuesta y texto de respuesta del administrador.
 - **Datos_Empresa:** datos fiscales del emisor (nombre, CIF, dirección, IBAN, logotipo) para PDFs y notificaciones.
 - **Evento:** actividades públicas (conciertos, actuaciones) gestionables desde el panel de admin.
 - **Token_Accion:** tokens de un solo uso (48h de validez) para aceptar/rechazar presupuestos por correo sin iniciar sesión.
 - **Keepalive:** tabla de utilidad para mantener activa la base de datos en Supabase.
-

4. Estructura general de navegación

La aplicación se organiza en las siguientes rutas:

- `/` — Inicio: hero, sección "Cómo funciona", eventos destacados y CTA de contacto.
- `/servicios` — Catálogo de equipos y DJs con filtros, disponibilidad en tiempo real y carrito.
- `/carrito` — Resumen de reserva, selección de fechas, datos del cliente y envío.
- `/eventos` — Listado público de eventos con buscador, toggle próximos/todos y mapa.
- `/contacto` — Formulario de contacto público.
- `/login` y `/register` — Autenticación con email/contraseña y OAuth (Google, GitHub).
- `/actualizar-password` — Flujo de restablecimiento de contraseña por correo.
- `/mis-pedidos` — Área privada del cliente: presupuestos y facturas con descarga de PDF y pago online.
- `/pago/:id` — Pasarela de pago: importe parcial o total, Stripe Elements o efectivo.

- `/mis-datos` — Edición de perfil y cambio de contraseña.
 - `/admin` — Panel de administración (solo admin): Presupuestos, Facturas, Usuarios, Contactos, Empresa.
 - `/presupuesto-confirmado` — Página de confirmación tras aceptar/rechazar por token.
 - `/mantenimiento` — Página de mantenimiento activada automáticamente cuando la API devuelve 503.
-

5. Flujos principales del sistema

5.1 Flujo de contacto

1. El visitante rellena el formulario de contacto (nombre, email, asunto, tipo, descripción).
2. El sistema registra el contacto con fecha automática y estado `respondido = false`.
3. N8N envía acuse de recibo al visitante y notificación al administrador con botón directo al panel.
4. El administrador responde desde la sección Contactos del panel de administración.
5. El sistema registra la respuesta, marca `respondido = true` y N8N envía la respuesta al usuario por correo.

5.2 Flujo de reserva

1. El usuario o el administrador crea una reserva indicando fechas y ubicación.
2. Se asocian equipos y/o artistas. El sistema comprueba disponibilidad en tiempo real (stock y solapamiento de fechas).
3. El sistema genera automáticamente el presupuesto con sus líneas de detalle, base imponible e IVA (21%).

5.3 Flujo de presupuesto por token

1. N8N envía al cliente el presupuesto en PDF con dos botones: Aceptar y Rechazar (tokens de 48h, un solo uso).

2. El cliente hace clic en el enlace del correo sin necesidad de iniciar sesión.
3. El backend valida el token (no usado, no caducado), ejecuta la acción y marca el token como usado.
4. Si acepta: el presupuesto pasa a `aceptado_cliente` y N8N notifica al administrador.
5. El administrador confirma desde el panel: el presupuesto pasa a `aceptado`, se genera la factura automáticamente y N8N envía al cliente el correo con botón de pago.
6. Si rechaza: el presupuesto pasa a `rechazado` y N8N notifica a ambas partes.

5.4 Flujo de pago

1. El cliente accede a `/pago/:id` desde el botón del correo o desde su área privada.
2. Introduce el importe a pagar (parcial o total, validado contra el pendiente).
3. **Pago online (Stripe):** el backend crea un `PaymentIntent`; el cliente introduce los datos de tarjeta en Stripe Elements; al confirmar, Stripe envía el webhook `payment_intent.succeeded` al backend; el backend verifica la firma, comprueba idempotencia por `referencia_pago`, inserta el pago y actualiza el estado de la factura si se alcanza el total.
4. **Pago manual (efectivo/transferencia):** el administrador registra el pago desde el panel de administración con importe y método.
5. N8N notifica a cliente y administrador en cada pago parcial y al completarse la factura.

5.5 Flujo de gestión de eventos

1. El administrador crea, edita o elimina eventos desde el panel de administración.
2. Los eventos son visibles públicamente en `/eventos` sin necesidad de autenticación.
3. Cada evento muestra título, fecha, lugar, artistas, descripción, imagen y ubicación en mapa (Google Maps).

6. Panel de administración

El panel de administración (`/admin`) está restringido a usuarios con rol `admin` y se organiza en cinco secciones:

- **Presupuestos:** listado con buscador y filtros por estado; cards expandibles con líneas de detalle; botones Aceptar/Rechazar; descarga de PDF; botón "Ver factura" con scroll y highlight automático.
- **Facturas:** listado con buscador y filtros por estado y método de pago; barra de progreso de pago; historial de pagos; formulario de registro de pagos manuales con modal de confirmación; descarga de PDF.
- **Usuarios:** listado de todos los usuarios registrados con nombre, email, teléfono y rol.
- **Contactos:** listado de mensajes recibidos con estado (sin responder / respondido); formulario de respuesta inline; botón en el correo de notificación que navega directamente al contacto con efecto de highlight.
- **Empresa:** formulario de edición de datos fiscales (nombre, CIF, dirección, teléfono, email, IBAN) y subida de logotipo.

La navegación entre secciones admite parámetros URL (`?contacto=ID` , `?presupuesto=ID`) para navegar directamente a un registro específico con scroll y efecto de highlight amarillo.

7. Coherencia con el modelo conceptual

El diseño del sistema garantiza que:

- Un usuario puede realizar una o varias reservas.
- Una reserva puede incluir uno o varios equipos y/o artistas.
- Cada reserva genera un único presupuesto.
- Cada presupuesto genera una única factura.
- Cada factura puede tener uno o varios pagos asociados.
- La entidad Contacto permite registrar y gestionar comunicaciones desde la navegación pública sin formar parte del flujo de reserva.
- La entidad Evento permite publicar actividades públicas sin relación con el flujo de reservas.
- La entidad Token_Accion desacopla la aceptación/rechazo de presupuestos de la sesión del usuario.

Esta estructura asegura la integridad de los datos y permite una gestión completa del ciclo de vida de una reserva, desde la solicitud hasta el cobro final.